

System Architecture Document

Rwanda NBSAP Monitoring Dashboard 2025–2030

Prepared for: Rwanda Environment Management Authority (REMA) **Document Reference:** NBSAP-SAD-2026-005 **Version:** 1.0 **Date:** June 2026 **Classification:** Official — Government Use

Document Control

Field	Value
Document Title	System Architecture Document
Project	Rwanda NBSAP Monitoring Dashboard
Prepared by	NBSAP Dashboard Development Team
Approved by	REMA — Department of Biodiversity & Landscape Management
Status	Final

1. Executive Architecture Overview

The Rwanda NBSAP Monitoring Dashboard follows a **serverless Backend-as-a-Service (BaaS)** architecture. This approach eliminates the need for Rwanda to provision, manage, or maintain backend servers, while providing enterprise-grade security, real-time capabilities, and global availability.

1.1 Architecture Principles

Principle	Rationale
Serverless-first	Eliminates server maintenance burden; pay-per-use cost model
Security by default	Row-Level Security on all tables; JWT authentication; HTTPS everywhere
Automation over manual	Database triggers compute metrics — no human calculations required
Real-time visibility	WebSocket push ensures all dashboards reflect current state
Global availability	CDN-distributed frontend loads fast from any location

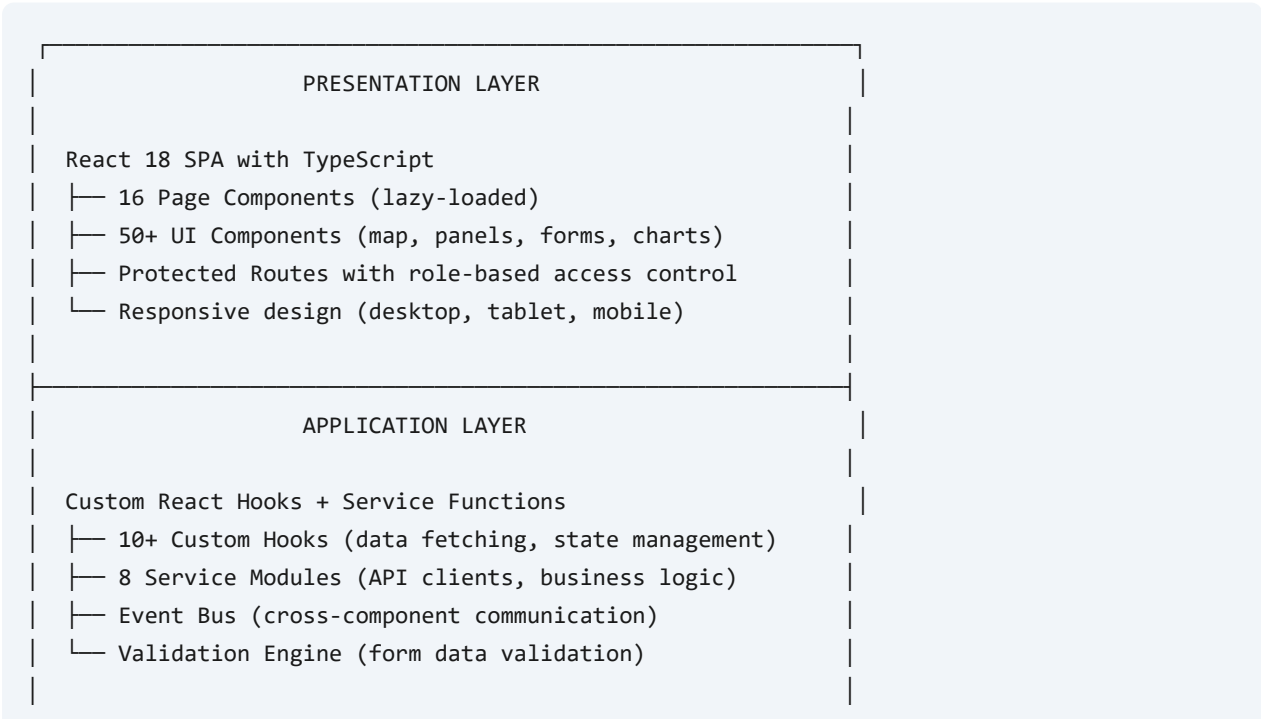
Cost efficiency	Free/low-tier cloud services support Rwanda's current user base
-----------------	---

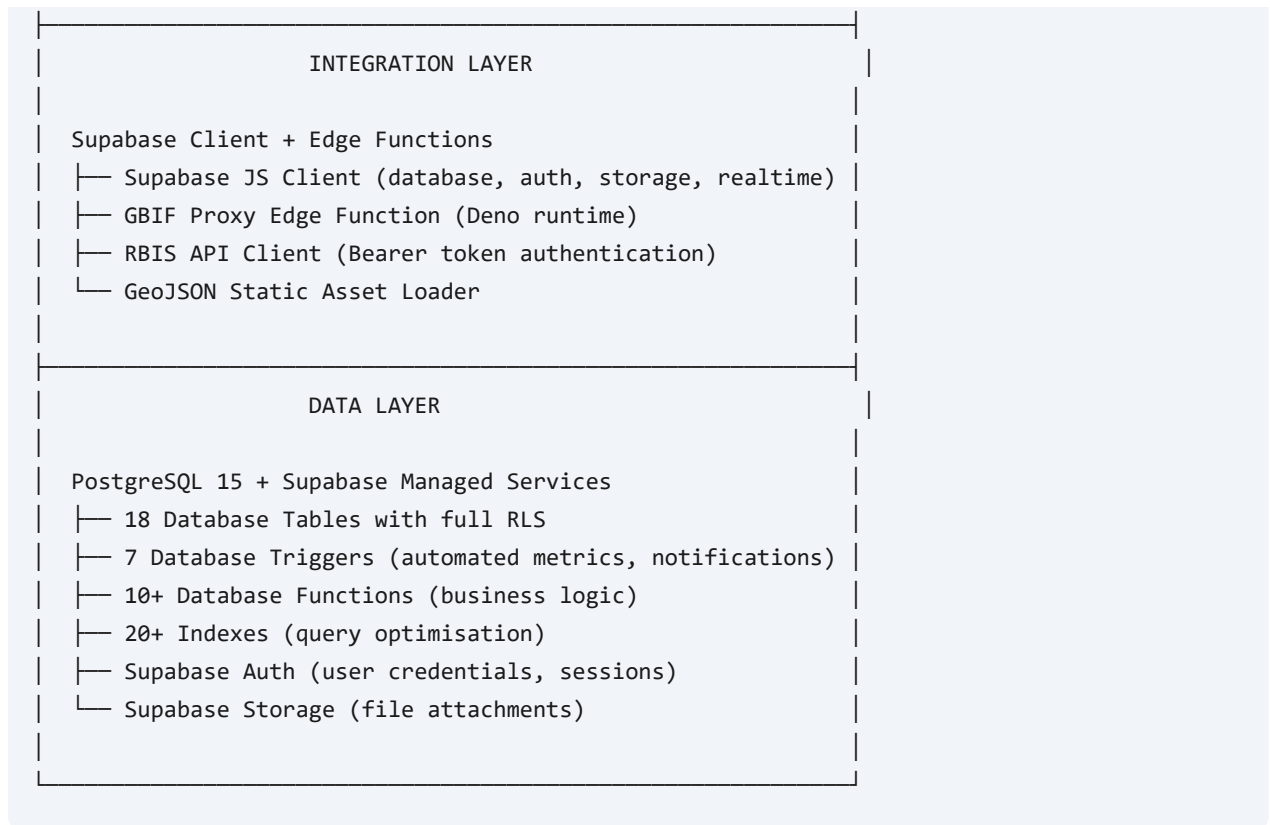
1.2 Architecture Decision Record

Decision	Options Considered	Chosen	Rationale
Backend approach	Custom Node.js API / Django / BaaS	Supabase (BaaS)	No server management; built-in Auth, RLS, Realtime, Storage
Frontend framework	React / Vue / Angular	React 18	Largest ecosystem; component reuse; strong TypeScript support
Build tool	Webpack / Vite	Vite	Faster builds; native ES module support; simpler configuration
Database	MySQL / MongoDB / PostgreSQL	PostgreSQL 15	RLS support; JSONB for flexible report data; proven at scale
Hosting	AWS / Azure / Vercel	Vercel	Git-push deploy; global CDN; free tier; optimised for React SPAs
Map library	Google Maps / Mapbox / Leaflet	Leaflet.js	Open source; no API key required; lightweight; mature ecosystem
Chart library	D3.js / Chart.js / Recharts	Recharts	React-native; declarative API; responsive; good defaults

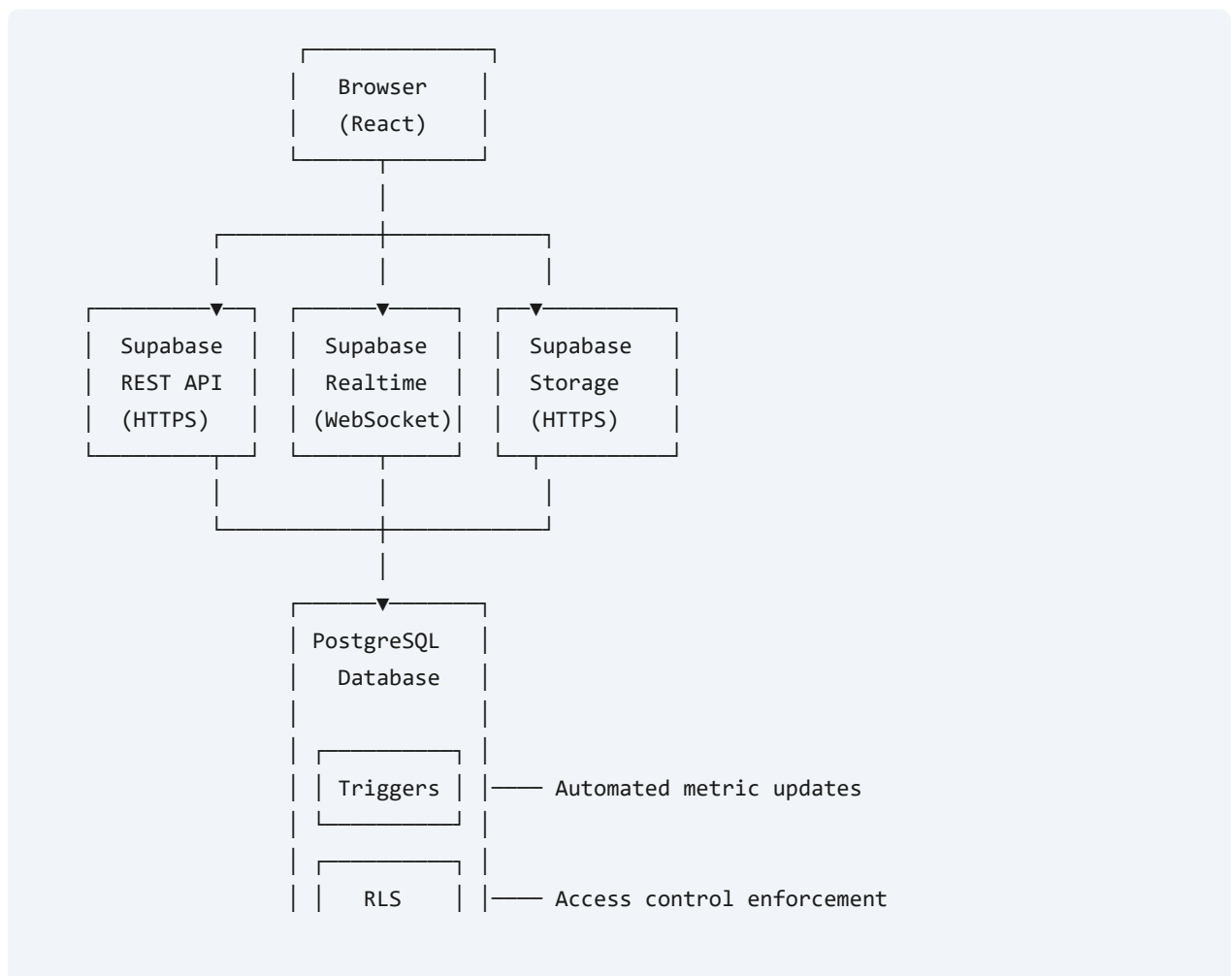
2. Logical Architecture

2.1 Four-Layer Model



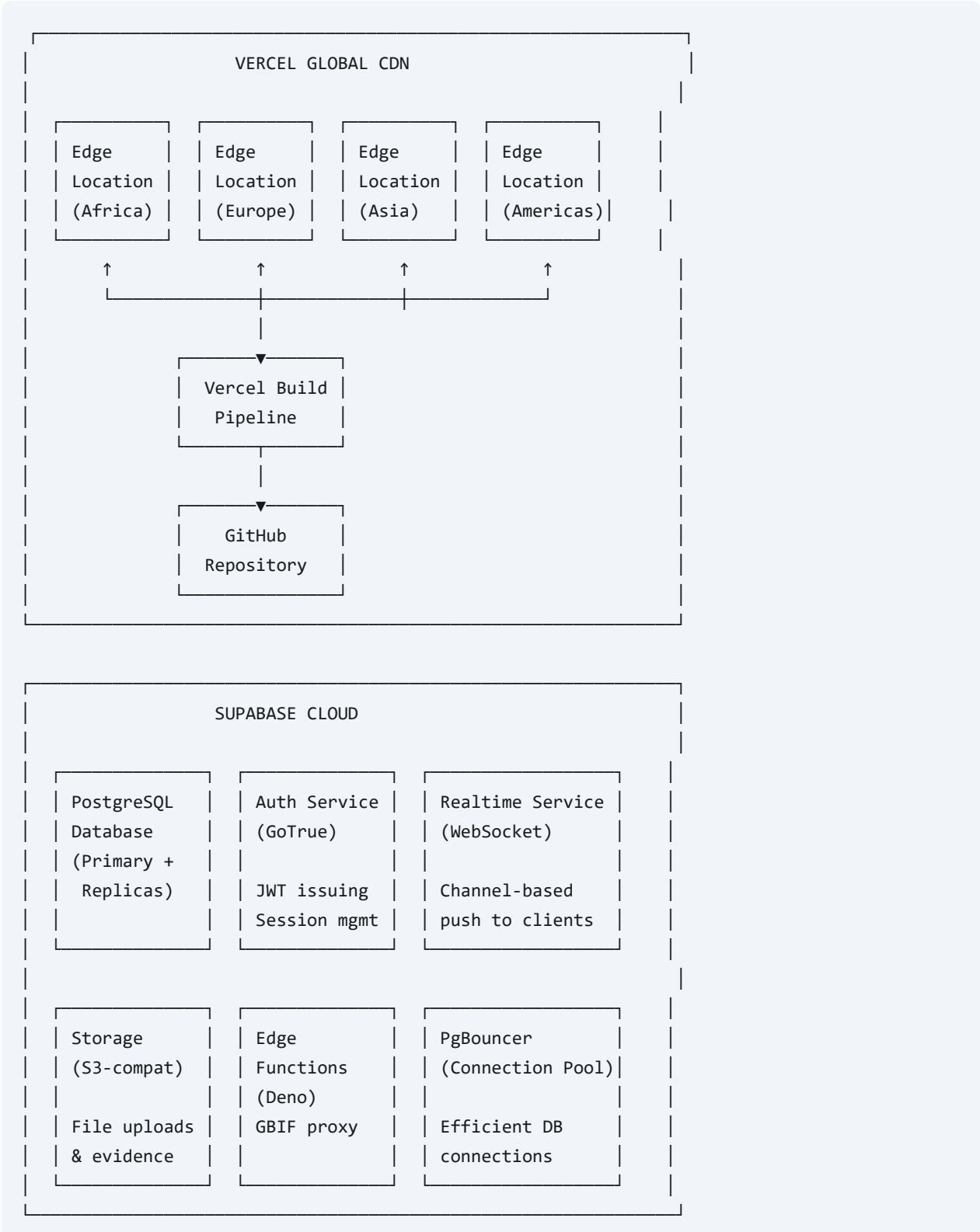


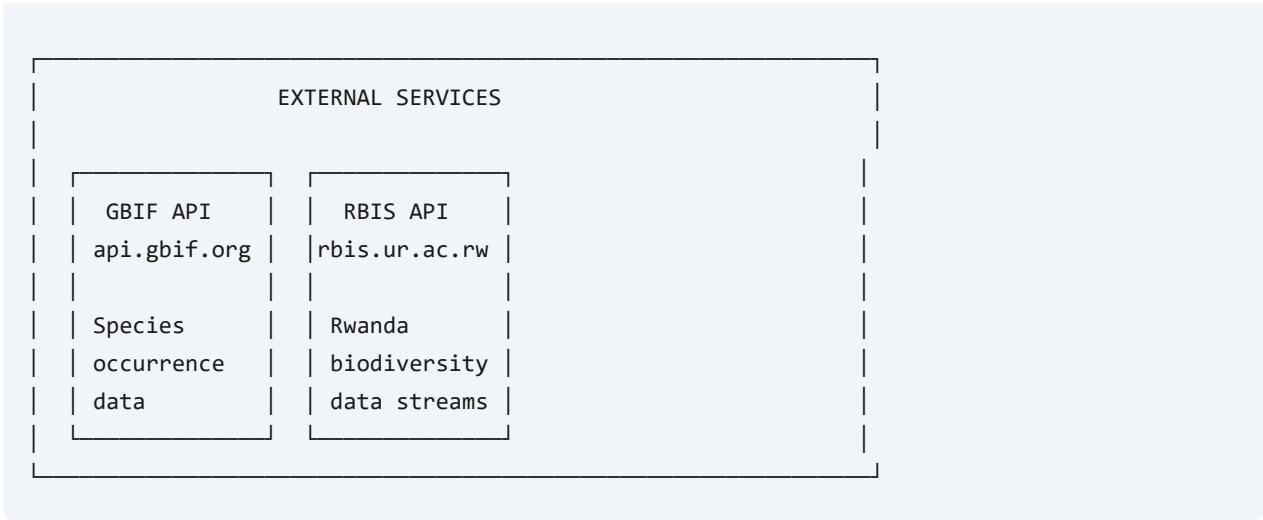
2.2 Component Interaction Diagram



3. Physical Architecture

3.1 Infrastructure Topology



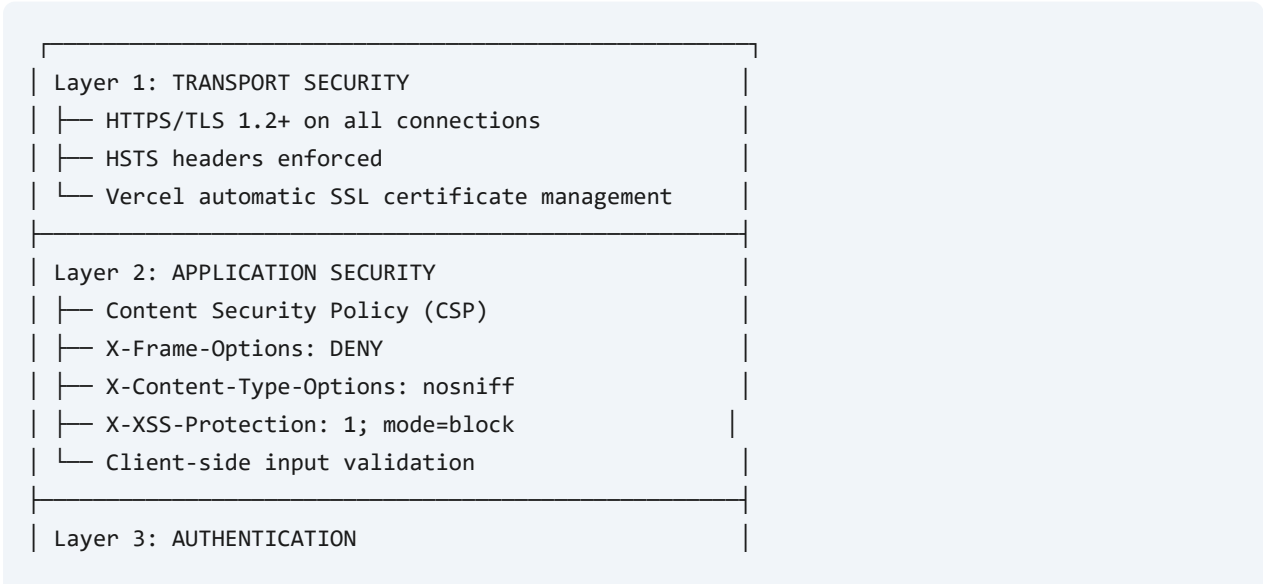


3.2 Network Flow

Flow	Protocol	Source	Destination	Port	Authentication
Browser → Frontend	HTTPS	End user	Vercel CDN	443	None (public)
Browser → Supabase API	HTTPS	End user	Supabase Cloud	443	JWT (anon key + session)
Browser → Supabase Realtime	WSS	End user	Supabase Cloud	443	JWT session
Edge Function → GBIF	HTTPS	Supabase	api.gbif.org	443	None
Browser → RBIS	HTTPS	End user	rbis.ur.ac.rw	443	Bearer token

4. Security Architecture

4.1 Security Layers



└─ JWT-based session management (Supabase Auth)
└─ bcrypt password hashing
└─ Automatic token refresh
└─ Account suspension/deactivation controls
Layer 4: AUTHORISATION
└─ 6 user roles with scoped permissions
└─ Protected routes (client-side enforcement)
└─ Row-Level Security (server-side enforcement)
└─ 25+ RLS policies on 18 tables
Layer 5: AUDIT & ACCOUNTABILITY
└─ Immutable append-only audit log
└─ Every action logged with user ID + timestamp
└─ Verification gate (data quality firewall)
└─ Role change request audit trail

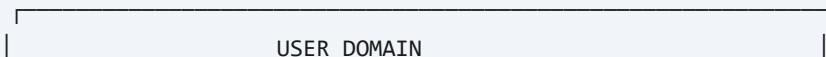
4.2 Data Flow Security

Reporter submits data

- └─ Client-side validation (TypeScript types + form rules)
- └─ HTTPS transport (TLS 1.2+)
- └─ JWT authentication verified (Supabase Auth)
- └─ RLS policy check (INSERT allowed for reporter role?)
- └─ Database constraint check (NOT NULL, CHECK, FK)
- └─ Data stored with status='pending'
 - └─ NO effect on national metrics until approved
- └─ Audit log entry created
- └─ Notification dispatched to reviewers

5. Data Architecture

5.1 Data Domain Model



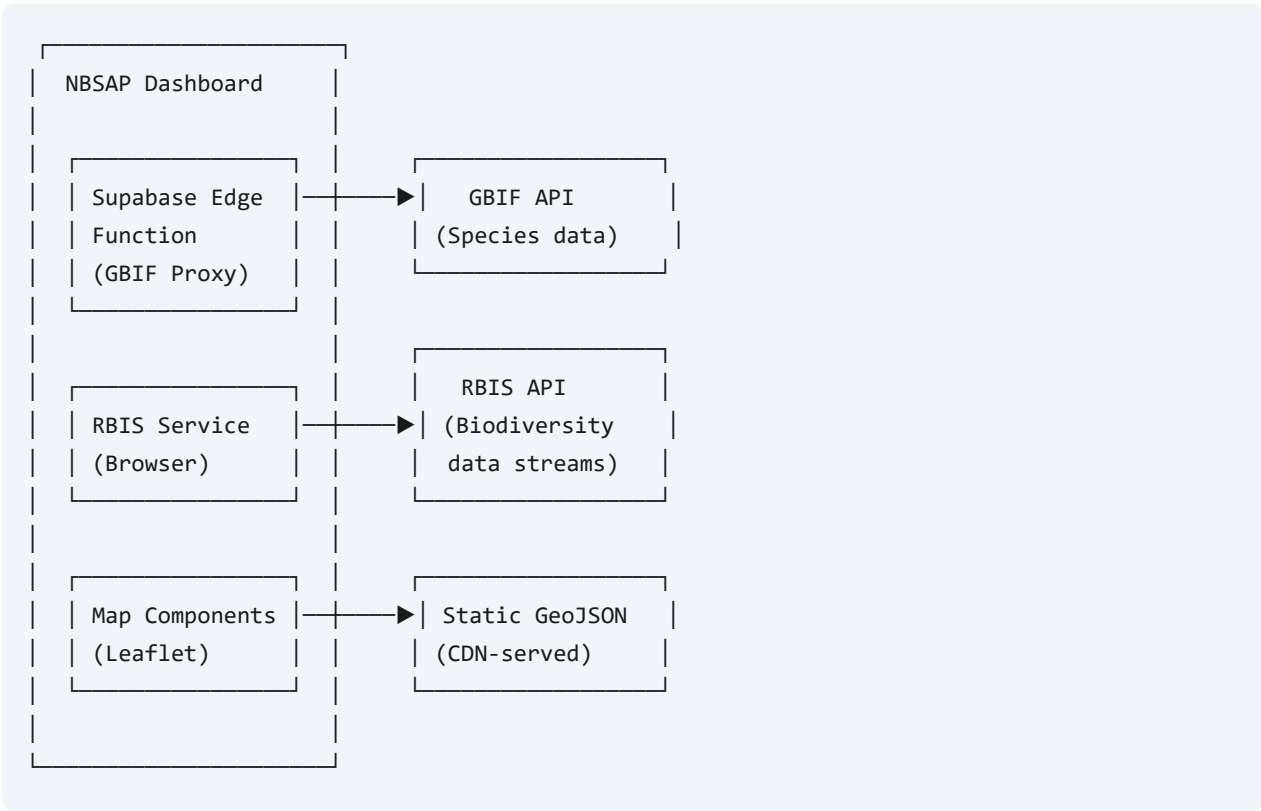


5.2 Data Lifecycle

Phase	State	Visibility	Metric Impact
Submitted	pending	Reporter + Reviewers	None
Under Review	pending	Reporter + Reviewers	None
Approved	approved	All authorised users	Metrics auto-updated
Rejected	rejected	Reporter + Reviewers	None
Deleted (if was approved)	Removed	N/A	Metrics auto-reversed

6. Integration Architecture

6.1 Integration Points



6.2 Integration Resilience

Integration	Failure Mode	System Behaviour
GBIF API down	Timeout after 30s	Dashboard shows cached data; map shows last-known occurrences
RBIS API down	Connection error	Status indicator turns red; manual data entry continues
Supabase down	Service unavailable	Full system offline; users see error page
GeoJSON CDN miss	404 error	Map renders without affected layer; console warning

7. Availability & Disaster Recovery

7.1 Availability Design

Component	SLA	Redundancy
Vercel CDN	99.99%	Multi-region edge network

Supabase Database	99.9%	Managed PostgreSQL with daily backups
Supabase Auth	99.9%	Managed GoTrue service
GBIF API	Best-effort	Graceful degradation; data cached
RBIS API	Best-effort	Dashboard functional without RBIS

7.2 Backup Strategy

Data	Method	Frequency	Retention
Database	Supabase automatic backup	Daily	7 days (free tier) / 30 days (pro)
Source code	GitHub repository	Every commit	Indefinite (git history)
Migration files	Git-versioned SQL files	Every change	Indefinite
GeoJSON data	Git-versioned static files	Infrequent	Indefinite
Environment config	Documented in handover docs	Manual	Project documentation

7.3 Recovery Procedures

Scenario	Recovery Action	Estimated Time
Frontend deployment issue	Vercel auto-rollback to previous deployment	< 5 minutes
Database corruption	Restore from Supabase daily backup	1–4 hours
Credential compromise	Rotate Supabase keys; redeploy with new <code>.env</code>	30 minutes
Complete infrastructure loss	Re-deploy from GitHub + run migrations on new Supabase project	2–4 hours

8. Scalability Considerations

8.1 Current Capacity

Dimension	Current Design	Headroom
Concurrent Users	500	Sufficient for current stakeholder base (~200 active users)
Report Storage	10,000+	Ample for quarterly reporting from 30 districts over 5 years
Database Size	Supabase free tier (500MB)	Current usage well under limit

API Calls	Supabase free tier (500K/month)	Current usage well under limit
File Storage	Supabase free tier (1GB)	Adequate for evidence attachments

8.2 Scaling Path

Should usage grow beyond current capacity:

Growth Trigger	Scaling Action	Cost Impact
> 500 concurrent users	Upgrade Supabase to Pro tier	\$25/month
> 500MB database	Supabase Pro (8GB included)	Included in Pro
> 1GB file storage	Supabase Pro (100GB included)	Included in Pro
Regional expansion	Supabase project in additional region	Separate instance
Mobile application	React Native app connecting to same Supabase backend	Development cost only

9. Technology Lifecycle

9.1 Technology Roadmap

Technology	Current	End-of-Support Risk	Upgrade Path
React 18	Active (LTS)	Low (2027+)	React 19 when stable
TypeScript 5	Active	Very Low	Incremental version upgrades
Vite 4	Active	Low	Vite 5/6 (non-breaking)
PostgreSQL 15	Active (LTS)	Very Low (2027+)	Supabase manages upgrades
Node.js 18	Active (LTS until 2025-04)	Moderate	Node.js 20/22 LTS
Leaflet.js	Stable	Very Low	Active open-source project

9.2 Dependency Management

- `package-lock.json` ensures reproducible builds
- `npm audit` identifies known vulnerabilities
- Quarterly dependency review recommended
- Supabase client library updates tracked

